



From Zero to Agentic: Building Your First AI-Driven Threat Investigation in Splunk Enterprise Security

SEC1215

David Pollard
Unshakeable Salt Ltd, @UnshakeableSalt

From Zero to Agentic: Building Your First AI-Driven Threat Investigation in Splunk Enterprise Security

Agentic AI promises autonomous security operations — but where do you actually start?

This session cuts through the hype with hard-won lessons from building a bespoke agentic use case inside Splunk Enterprise Security from scratch.

We cover LLM selection on a real-world budget, native Splunk integration patterns, and the critical pitfalls that cost time before delivering value. Practical, honest, and immediately applicable.



Takeaway #1

Choose the right LLM for SecOps — balancing cost, latency, privacy, and capability without defaulting to the biggest name.



Takeaway #2

A repeatable pattern to integrate agentic workflows into Splunk ES — what to build yourself vs. what Splunk gives you for free.



Takeaway #3

Avoid the top pitfalls — a prioritised starting framework for teams with limited time and budget to reach value fast.

Agenda

- 01 The Agentic Promise
- 02 Choosing the LLM
- 03 A three-phase loop
- 04 Phase 1 – The “Alert Action”
- 05 Phase 2 – Bridge Mechanics & CIM
- 06 Phase 3 – Zero Touch
- 07 Demo
- 08 Recap & Pitfalls

'Agentic AI' is Easy to Say, - Hard to Ship

Every vendor keynote: autonomous SOC, self-healing detections, AI analysts.

The gap nobody shows you: what connects an LLM's opinion to a field Splunk can score, correlate, and dashboard.

- An LLM's reply is prose, or loose JSON at best — Enterprise Security's Risk Analysis framework and Incident Review don't care what the AI **said**. They care about **fields**.
- Start with Smart LLM's and tune down to specific API addressing.
- The plumbing between "*the AI had a thought*" and "the SOC analyst sees a risk score, with zero copy/paste."

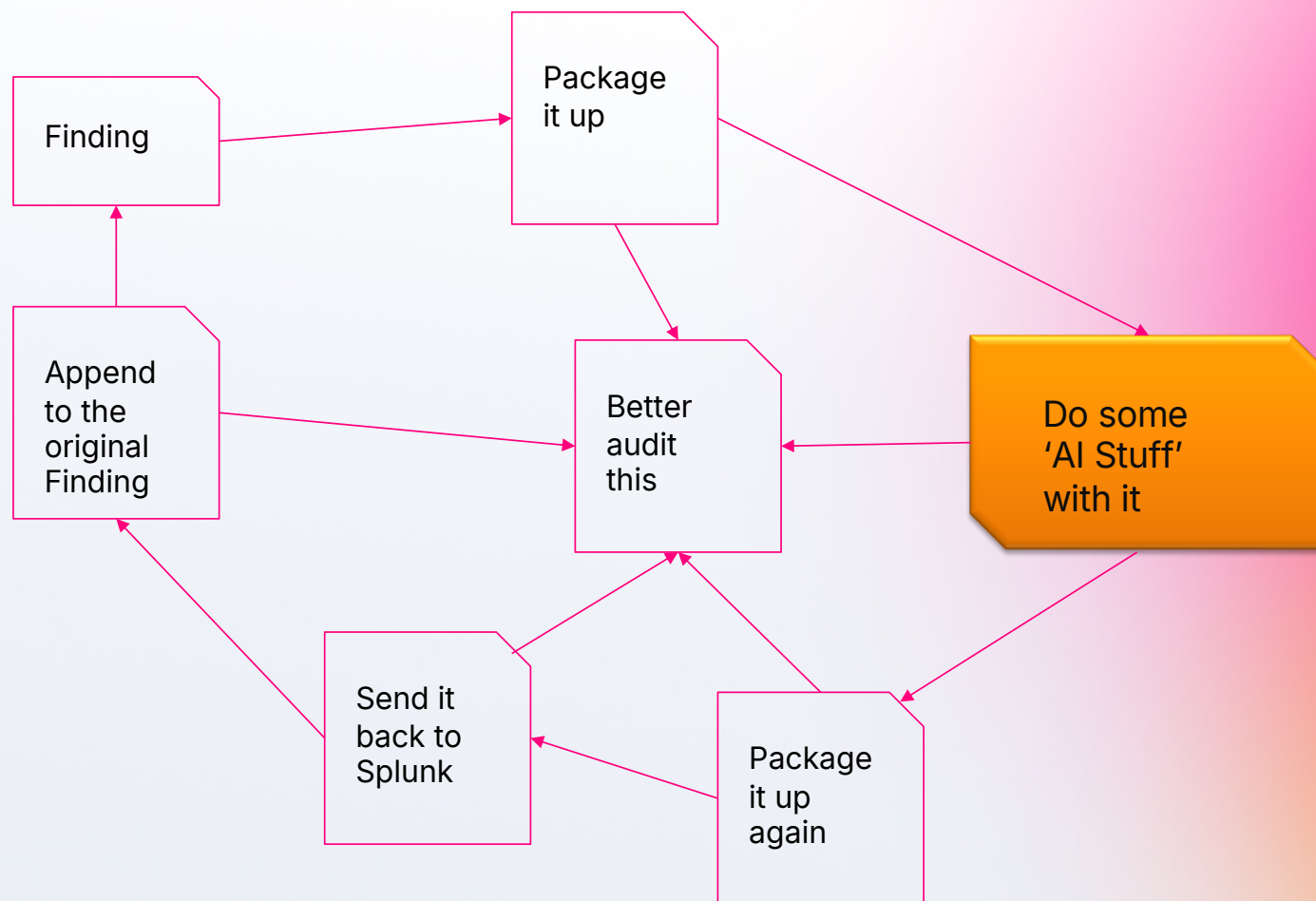
Proof of Concept

We have no SOAR, we have no AI

**Take a finding we see in Enterprise Security,
where we normally have to rely on a person
performing an action, manually gathering
some enrichment data, rekeying it into our
platform and waiting for the next decision
point.**

'Apply some Agentic AI magic'

Enterprise
Security



“

**Other LLM and AI Providers are
available...**

Is in fact Takeaway #1

Choosing the Right LLM for SecOps

Four axes that actually matter - not "which model tops the leaderboard":

Cost

Priced per notable/investigation, not per demo. A SOC generates volume; the biggest frontier model on every single notable gets expensive fast. Learning as we did this, treat the same as developing detections.

Latency

The call happens synchronously, in-line, inside the alert action itself. The correlation search's adaptive response doesn't finish until the model replies, so round-trip time is part of the alert pipeline's own latency budget, not a footnote.

Privacy

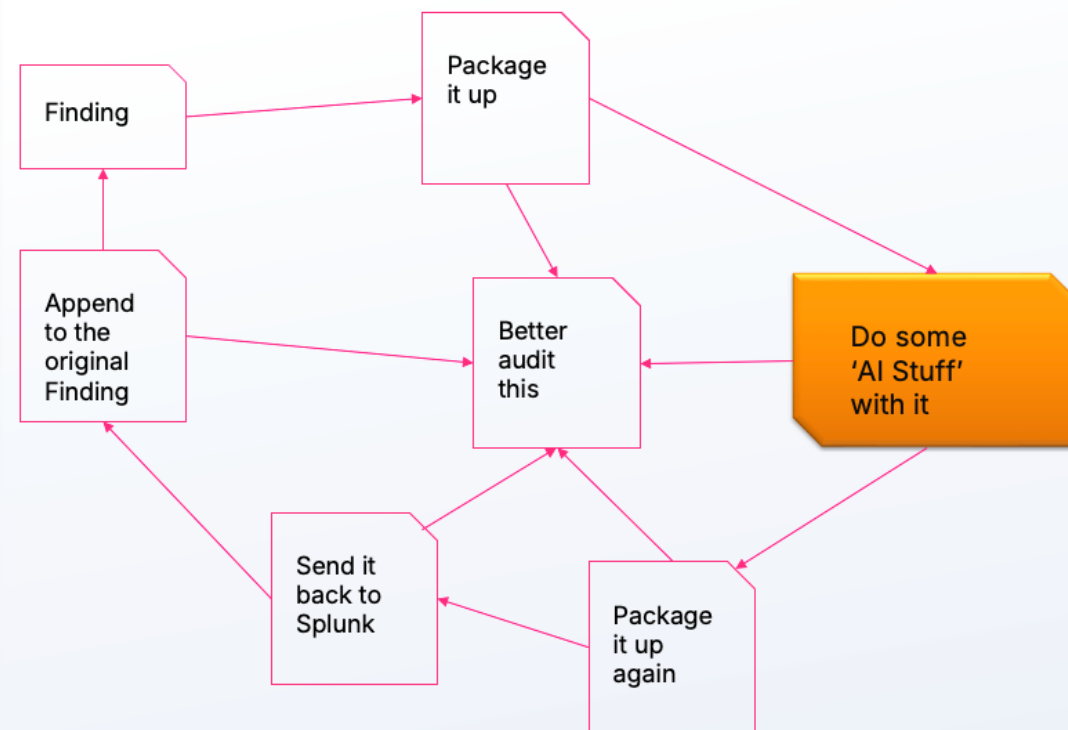
The payload is a full CIM/risk notable export. Where does that data go, and under what retention/training terms?

Capability

Does it need to reason over a full raw event, or just answer three standing yes/no/score questions well?

The Agentic Loop, One Synchronous Step

- ES Correlation Search
 - Triggers Notable/Risk event
 - Adaptive Response Action: TA_ResponseActions
 - Builds the full CIM/Risk notable as JSON
 - Answers an "AI Prompt" block of standing checks via Synchronous Call to AI API
- Receives answer back as an "AI Response"
- One payload back
 - Slack for Visual Audit
 - Originating notables audit trail
 - Notable enrichment KV Store record



Phase 1: ES Alert Action Feeds the Agentic AI

- Takes the triggering notable/finding and delivers it to Slack as a **complete JSON payload** every CIM/risk/notable field, noise fields stripped, no size-limited summary.
- Ships a standing **`perplexity\_ask`** block by default in **`param.additional\_fields`**:
 - three checks that travel with every notable
 - Is **`\$result.repo\$`** a repo we commonly see for this finding type?
 - Is **`\$result.user\$`** an expected/authorised user?
 - Is **`\$result.src\$`** a low-threat source IP?
 - Two delivery modes:
 - file_upload (bot token, no size limit)
 - webhook (quick setup, truncates large payloads).

```
1 {"perplexity_ask":
2   {"repository_check": "Check that $result.
3     repo$ is a repository we commonly see
4     associated with this type of finding.",
5     "user_check": "Check that $result.user$ is an
6     expected/authorized user for this kind of
7     activity.",
8     "source_ip_check": "Check that source IP
9     $result.src$ is low threat (not on any known
        malicious/blocklist sources)."}
}
```

What Splunk Gives You for Free vs. What You Build

Splunk gives you for free	You build
Adaptive Response framework	The delivery shaping (JSON payload, denylist)
`cim_actions.py`'s `ModularAction` - native panel reporting	The standing-question payload (`perplexity_ask` block)
The Perplexity Chat Completions API's `response_format` (JSON Schema)	The synchronous in-line call + the schema shape itself
The Notable Event API + KV store	The KV-store enrichment + custom drilldown dashboard

The pattern, once built, is reusable per correlation search - you only write the glue once.

Phase 2, Take 1: What We Thought We Needed

- **The fix, at the time:** pin the required JSON shape into the prompt itself
- (``perplexity_ask.reply_format`` in ``alert_actions.conf``), so the reply is deterministic, not scraped from free text. It worked and it's not what ships today.
- Here's why we built it, and exactly how it worked.

AI verdict concept	CIM Risk field it becomes
Which user/host/IP is this about	<code>`risk_object`</code> <code>`risk_object_type`</code>
How risky is this, on our scale	<code>`risk_score`</code>
Why — the AI's reasoning, in one line	<code>`risk_message`</code>
Which notable this ties back to	<code>`orig_sid`</code> <code>`orig_rid`</code> <code>`event_id`</code>

Phase 3, Take 2: One Payload, Three Existing Write-Back Paths

No bespoke store, and no second process either.

The same envelope that carried ``perplexity_ask`` now also carries ``perplexity_response``, straight into the three paths that our Splunk App already had:



Slack

Still useful for visually auditing exactly what was asked and what came back, per notable, as it happens.



The Notable Event API → the originating notable

``POST /services/notable_update`` comment summarising what was sent, now including the Perplexity answer - visible directly in that notable's own Activity timeline in Incident Review.



The ``notable_enrichment`` KV store & custom drilldown

The full ask-and-response JSON as a structured record, reached from the notable's Adaptive Responses panel.

No analyst forwards, tags, or re-keys anything. All three writes happen inside the same alert-action invocation — no waiting on a reply, no second component to trigger them.

Live DEMO

LIVE DEMO SCRIPT

- **TRIGGER:** fire a correlation search in Splunk ES; a notable lands in Incident Review.
- **FIRE THE ACTION:** `TA\_ResponseActions` builds the full CIM/risk JSON, including the `perplexity\_ask` standing checks.
- **The AI answers, in-line:** before anything is sent anywhere, a single synchronous call to the Perplexity API returns a structured `perplexity\_response`, merged straight into the same payload.
- **One payload, three deliveries:** Slack, the notable's comment trail, and the KV store record all get written in the same action invocation. No polling, no wait.
- **Payoff:** show the same notable in Incident Review: Adaptive Responses panel, notable comment, and the KV-store drilldown dashboard, all populated

```

{ "source": "splunk_es_adaptive_response",
  "action": "notable_to_perplexity_json",
  "sent_at": "2026-08-03T10:31:36Z",
  "search_name": "Test Harness Sample Finding",
  "sid": "test-harness-notable_to_perplexity_json-1785753096",
  "app": "search",
  "owner": "admin",
  "results_link": "",
  "server_host": "localhost",
  "notable": {
    "event_id": "5B2F1B0A-0000-0000-0000-TESTHARNESS01",
    "rule_name": "Test Harness Sample Finding",
    "search_name": "Test Harness Sample Finding",
    "repo": "UnshakeableSaltLtd/SplunkConf",
    "user": "dpollard",
    "src": "203.0.113.42",
    "dest": "10.0.0.5",
    "signature": "Sample Finding (test_harness.py)",
    "urgency": "medium"
  },
  "additional_fields": {
    "perplexity_ask": {
      "repository_check": "Check that UnshakeableSaltLtd/SplunkConf is a repository we commonly see associated with this type of finding.",
      "user_check": "Check that dpollard is an expected/authorized user for this kind of activity.",
      "source_ip_check": "Check that source IP 203.0.113.42 is low threat (not on any known malicious/blocklist sources).",
    },
    "perplexity_response": {
      "repository_check": "UnshakeableSaltLtd/SplunkConf is a private internal repo owned by the organization, not a public/third-party repo commonly linked to this finding type; no reputation concerns found.",
      "user_check": "dpollard is a recognized internal user with prior legitimate activity against this repository; no anomaly detected for this action type.",
      "source_ip_check": "203.0.113.42 falls in the TEST-NET-3 documentation range (RFC 5737) and is not a real routable/malicious address; treat as low threat only because it is a non-production placeholder IP.",
      "overall": "No indicators of compromise: the repo and user are both expected/authorized, and the source IP is a benign documentation-range placeholder rather than a real threat source. Low risk overall; no analyst follow-up required beyond routine review."
    }
  }
}

```

Pitfalls We Hit

Pitfall 1: Notable fields

``/services/notable_update``
only accepts ``comment`` / ``status`` / ``urgency`` /
``newOwner`` / ``disposition``.

There is no REST call that attaches a new indexed field to an existing notable after the fact — "notable enrichment" has to fall back to a KV-store + custom-drilldown pattern, plus a plain-text comment for anything visible directly in Incident Review.

Pitfall 2: Silent Failures

Our first designs had to read a reply back out of Slack, and Incoming Webhooks carry no bot token to do that with — a correlation search wired to ``delivery_method=webhook`` still delivered fine, it just never got a verdict, and nothing errored to tell you.

Current designs don't read anything back out of Slack at all, so this entire failure class is gone (not fixed, but removed), by trusting the alert action's own synchronous context instead of a second read path.

Pitfalls We Hit

Pitfall 3: Solved once, then solved properly: LLM reply-format drift.

Free-text answers to the original standing checks were unparseable.

The first fix: pin the required JSON shape into the **prompt itself** (``perplexity_ask.reply_format``) and have the bridge **log-and-skip, not crash**, on anything that still didn't parse.

It worked, but it was still prose the model **chose** to format correctly.

Later fixes go one layer deeper: the Perplexity API's own ``response_format`` (JSON Schema) constrains the output structurally - there's no prose to parse, and no reply-format instruction competing for prompt space with the actual questions.

Pitfalls We Hit

Pitfall 4: Scripted Input overheads

Our initial build used a plain classic scripted input,

- Not a full `splunklib.modularinput` scheme
- No auto-generated Splunk Web settings page for its custom keys
- Required a hand-edited `local/inputs.conf`

That trade-off didn't get fixed with a better input scheme; the team looked at the whole component and decided the poller itself wasn't earning its keep.

This is the single clearest "we reacted fast once we saw the operational cost" moment in the whole build. Go back to Takeaway #2 and see what Splunk provides for you 'out of the box'.

Pitfalls We Hit

Pitfall 5: Change

- Spanning numerous versions of ES, we found 'different' ways of displaying the same data.
- Fronter AI providers change their services, costs and libraries. Sometimes that bug isn't yours.

From Zero to Agentic: Building Your First AI-Driven Threat Investigation in Splunk Enterprise Security



Takeaway #1

There's no such thing as being capable, quick, private and cheap.

Leverage what you have and what you know.



Takeaway #2

Splunk provides so much integration for free.

Splunk Community is the most valuable resource

Keep it simple, supported, safe



Takeaway #3

Keep it extensible and expandable.

Make mistakes, learn fast and use your existing development pipelines to grow



From Zero to Agentic: Building Your First AI-Driven Threat Investigation in Splunk Enterprise Security

SEC1215

David Pollard
Unshakeable Salt Ltd, @UnshakeableSalt

Resources:
<https://github.com/UnshakeableSaltLtd/SplunkConf>

Thank you

